Document Intelligence Refinery – Interim Report

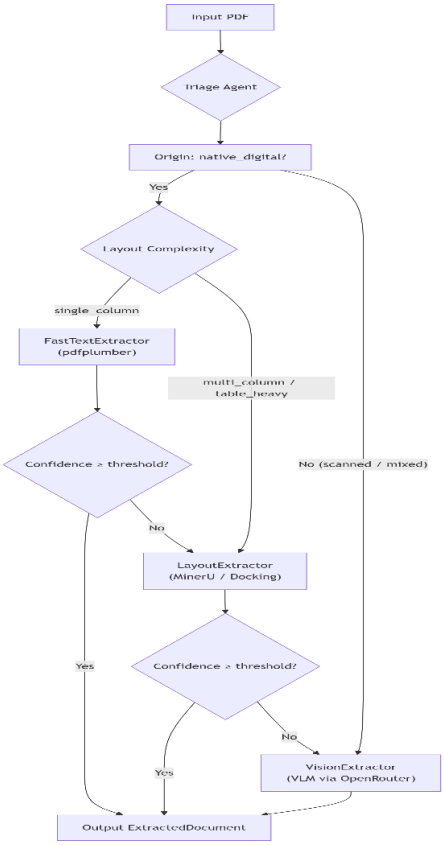
Date: March 4, 2026

Prepared by: Mikias Dagem

1. Domain Notes Deliverable

1.1 Extraction Strategy Decision Tree

The decision tree guides the selection of extraction strategy based on document characteristics determined by the Triage Agent. The tree incorporates confidence-gated escalation to ensure graceful degradation.



Confidence Thresholds (defined in extraction_rules.yaml):

- FastText: min character count 100, character density ≥ 0.01 chars/point², image area ≤ 50% of page.

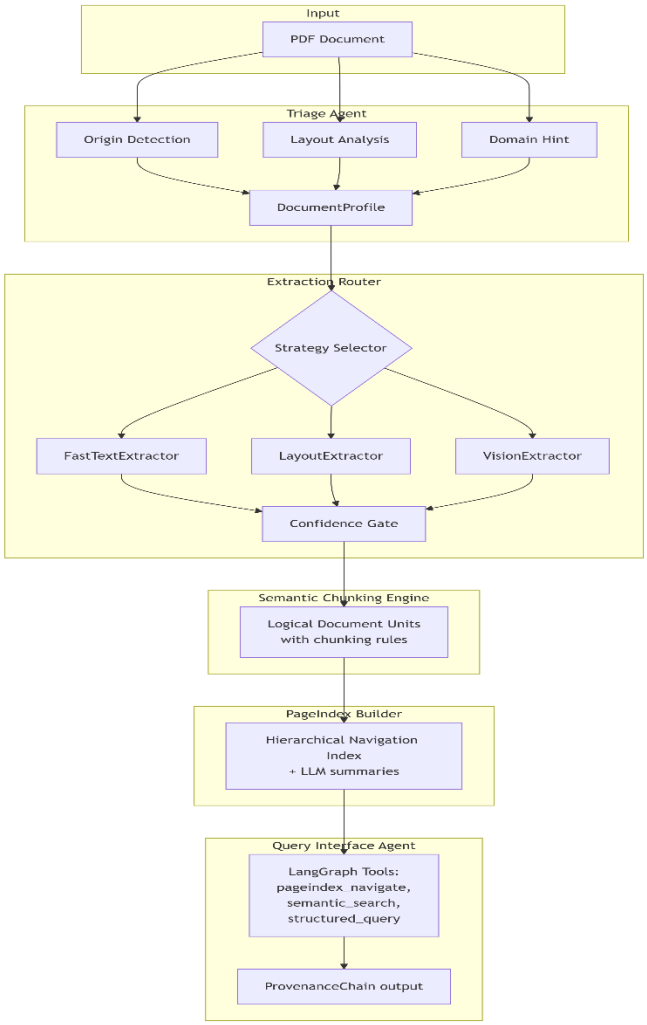
- Vision escalation threshold: overall confidence < 0.7 triggers next strategy.

1.2 Failure Modes Observed Across Document Types

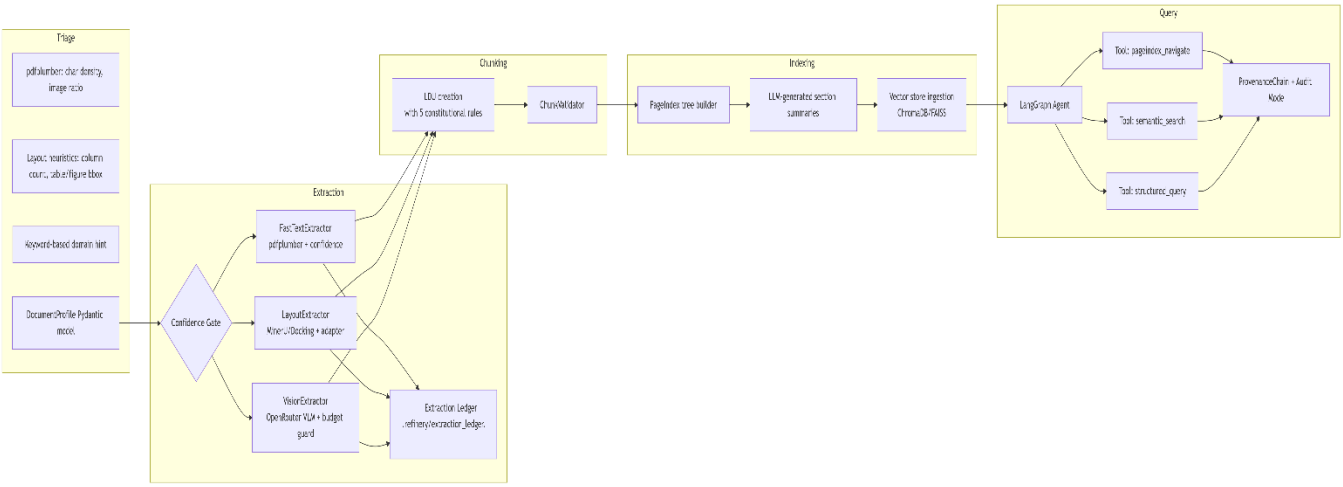
During initial exploration with the provided corpus (financial reports, scanned legal docs, technical assessments, tax expenditure tables), the following failure modes were identified:

Document Type	Failure Mode	Mitigation
Scanned PDF with	OCR produces gibberish or misses handwritten annotations	Escalate to VisionExtractor with a handwriting-aware prompt; use image preprocessing
Multi-column layout	Text reading order scrambled when using simple text extraction	Use LayoutExtractor (MinerU) that reconstructs columns via layout models
Nested or spanning tables	Table structure flattened, losing hierarchical relationships	Store tables as JSON with nested representation; merge chunks with identical header signatures across pages
Low-resolution scans	Character recognition errors cause low confidence	Apply image enhancement (sharpening, contrast) before passing to VLM; if budget allows, use super-resolution
Mixed language documents	Language detection fails, leading to wrong extraction prompts	Use language detection model (e.g., `langdetect`) on first page; fallback to VLM with language identification instruction
Extremely large documents (>200 pages	Vision extraction costs explode	Implement budget guard: cap pages processed by VLM; switch to Tesseract OCR after limit with warning in ledger

1.3 Pipeline Diagram (High-Level)



2. Architecture Diagram – Full 5-Stage Pipeline with Routing Logic



Key Features:

- Strategy Pattern:Each extractor implements a common interface, allowing easy addition of new strategies.

- Confidence Gating: FastText and Layout extractors compute confidence; if below threshold, the router escalates to the next tier.

- Budget Guard: Vision extraction monitors token cost; if a document exceeds the configured budget, extraction falls back to a cheaper method (e.g., Tesseract) and logs a warning.

- Externalized Configuration: All thresholds and chunking rules are stored in rubric/extraction_rules.yaml, enabling adaptation to new domains without code changes.

3. Cost Analysis – Estimated Cost per Document by Strategy Tier

Cost estimates are based on typical document lengths (20–50 pages) and current API pricing for OpenRouter (as of March 2026). These figures are used by the budget guard to prevent cost overruns.

Strategy Tier	Tools Used	Estimated Cost per Document	When Applied
Strategy A – Fast Text	pdfplumber, pypdf	0.01 - $0.05 CPU only, no API calls)	Native digital PDFs with simple layout (single column, minimal tables/figures)
Strategy B – Layout-Aware	MinerU, Docking (local models)	$0.10 –$0.50 (local inference, no API)	Multi-column documents, table-heavy reports, mixed layout
Strategy C – Vision-Augmented	OpenRouter (GPT-4o, Gemini Pro Vision, etc.)	$0.02 – $0.10 per page $0.40 – $5.00 for 20–50 pages	Scanned documents, low-confidence extractions from A/B, handwriting present

Budget Guard Implementation:

- Configurable max_cost_per_document in extraction_rules.yaml (default $5.00).

- VisionExtractor tracks cumulative token cost; if exceeded, it stops processing further pages and falls back to OCR (Tesseract) with a warning in the ledger.

- All extraction attempts are logged in .refinery/extraction_ledger.jsonl with strategy_used, confidence_score, cost_estimate, and processing_time.

Repository Status (Interim)

The GitHub repository contains all required components for the interim submission:

- Core Models : src/models/ with fully defined Pydantic schemas: DocumentProfile, ExtractedDocument, LDU, PageIndex, ProvenanceChain.

- Triage Agent: src/agents/triage.py implements origin_type, layout_complexity, domain_hint classification using pdfplumber heuristics.

- Extraction Strategies: src/strategies/ includes FastTextExtractor, LayoutExtractor (with MinerU adapter), and VisionExtractor (OpenRouter). All share a common interface.

- Extraction Router:src/agents/extractor.py routes based on DocumentProfile, implements confidence-gated escalation, and writes to extraction_ledger.jsonl.

- Configuration: rubric/extraction_rules.yaml externalizes thresholds and chunking rules.

- Artifacts: .refinery/profiles/` (to be generated) and .refinery/extraction_ledger.jsonl (sample ledger format defined).

- Unit Tests: tests/test_triage.py and tests/test_confidence.py validate classification and confidence scoring.